Roma Tre

Information Visualization

Drawing Graphs with the Force Directed Paradigm

Part II

M. Patrignani

 copyright©2026 m.patrignani

Copyright notice

- All the pages/slides in this presentation, including but not limited to, images, photos, animations, videos, sounds, music, and text (hereby referred to as “material”) are protected by copyright
- This material, with the exception of some multimedia elements licensed by other organizations, is property of the authors and/or organizations appearing in the first slide
- This material, or its parts, can be reproduced and used for didactical purposes within universities and schools, provided that this happens for non-profit purposes
- Any other use is prohibited, unless explicitly authorized by the authors on the basis of an explicit agreement
- This copyright notice must always be redistributed together with the material, or its portions

 copyright©2026 m.patrignani

Credits

- Giuseppe Liotta (Perugia University)
  - provided me with his slides on force directed methods
- “The book”
  - [Di Battista *et al.*, 99]
    - G. Di Battista, P. Eades, R. Tamassia, and I. G. Tollis, “Graph Drawing: Algorithms for the Visualization of Graphs”, Prentice-Hall, 1999
    - Chapter 10 on force directed approach

 copyright©2026 m.patrignani

Summary

- Scalability of the force-directed paradigm
  - Fruchterman & Reingold recipe
  - Matsubayashi & Yamada variant of F&R
  - Barnes–Hut algorithm
- Flexibility of the force-directed paradigm
  - simulating graph theoretic distances
  - magnetic fields
  - generic energy functions
  - drawing constraints
- Concluding remarks

 copyright©2026 m.patrignani

Scalability of the Force-Directed Paradigm

 copyright©2026 m.patrignani

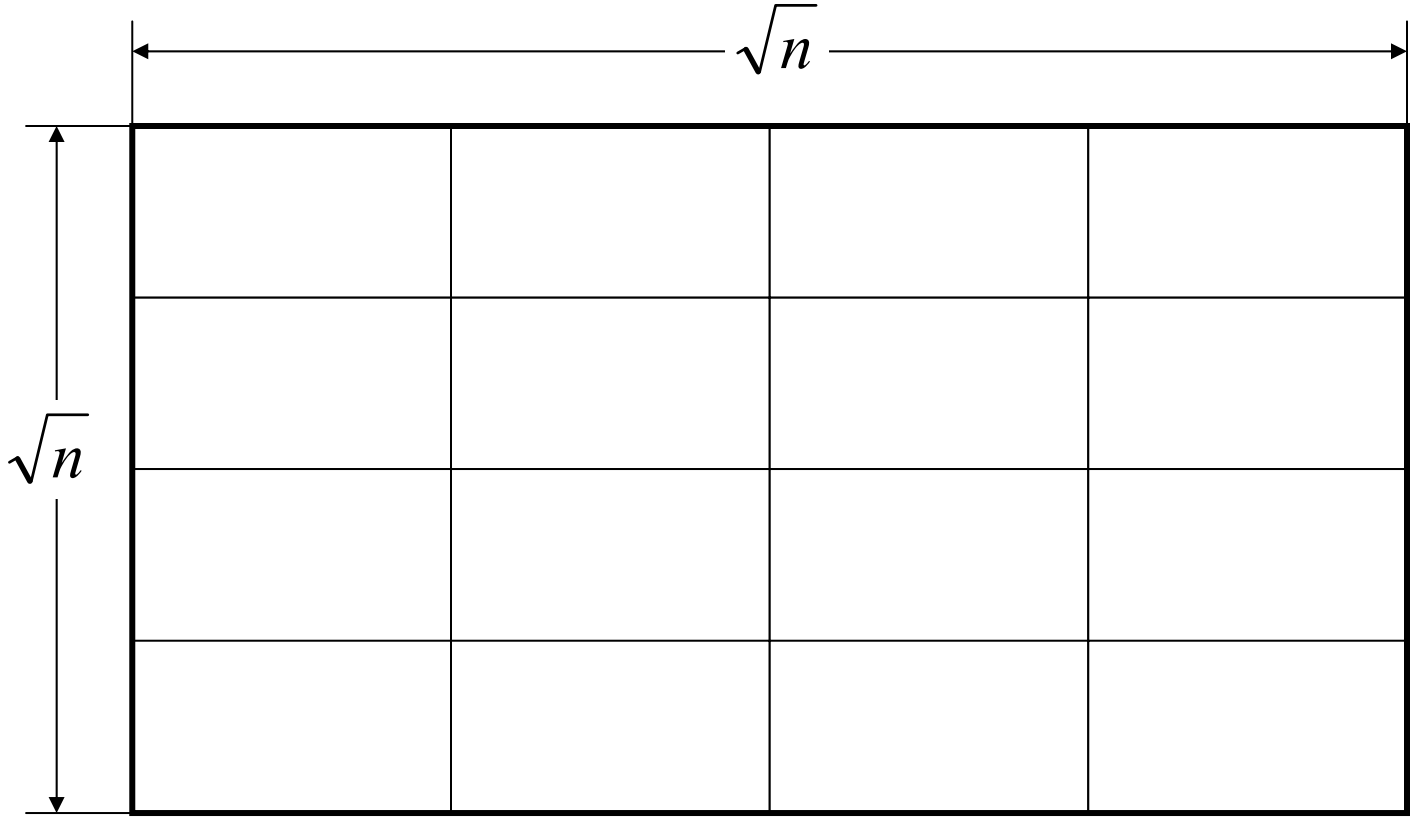
Where the complexity lies

- The heaviest phase of force-directed algorithms is the computation of the repulsive force
  - vertices must be paired in $O(n^2)$ ways
  - spring forces, instead, can be computed in O(n) if the graph is sparse and has O(n) edges
- We will see two possible approaches
  - Fruchterman & Reingold
    - plus two variants of it
  - Barnes & Hut

 copyright©2026 m.patrignani

Fruchterman & Reingold recipe

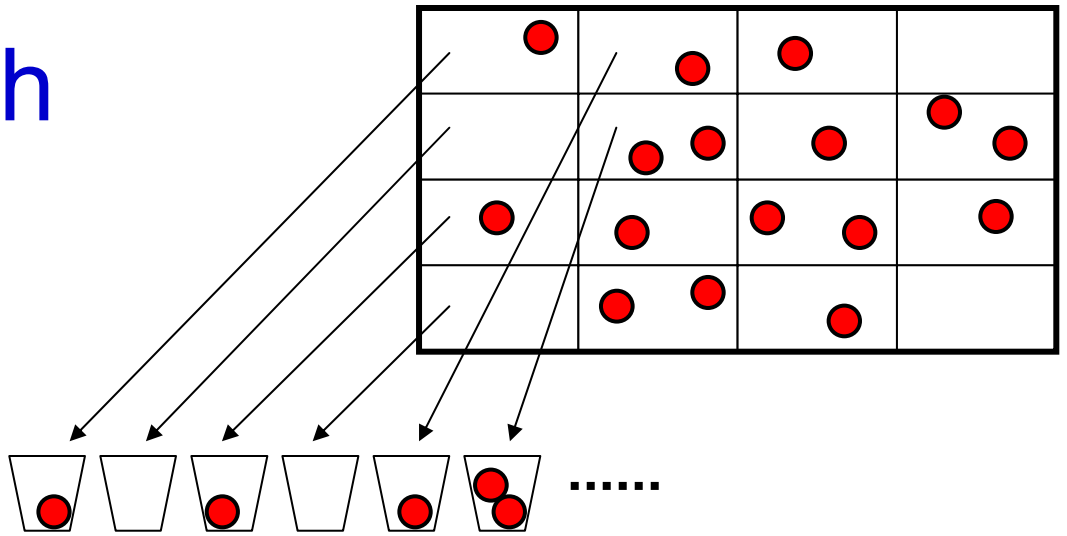
- Divide the drawing area into $\sqrt{n} \times \sqrt{n}$ rectangles



 copyright©2026 m.patrignani

Building buckets

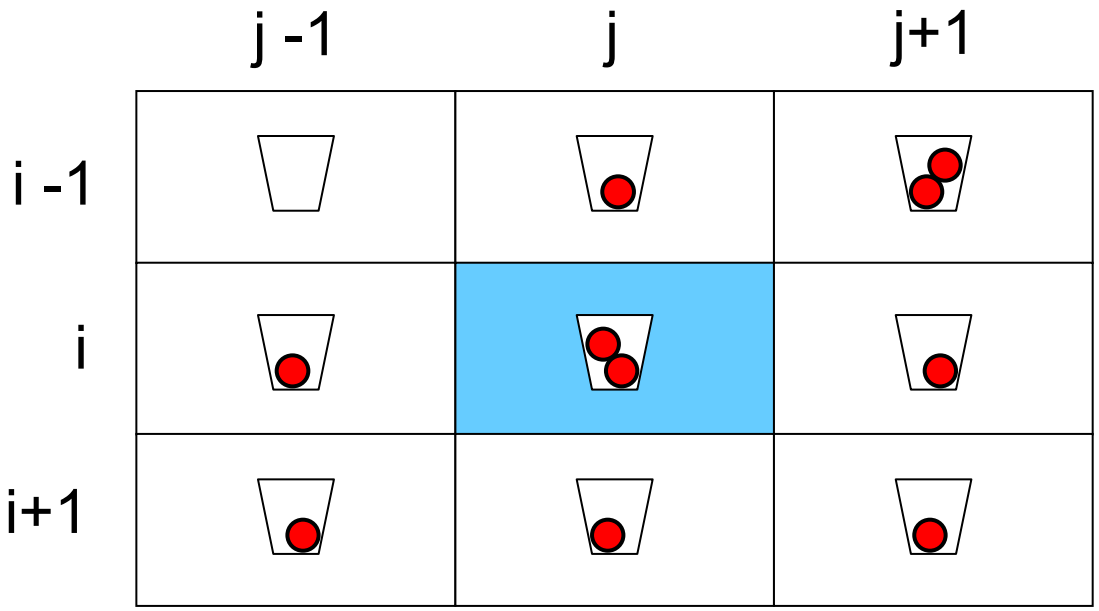
- The number of rectangles is $\sqrt{n} \times \sqrt{n} = n$
- If the vertices were uniformly distributed in the drawing area we would have O(1) vertices for each rectangle
- Associate a bucket with each rectangle
  - i.e., a set (or list) of the contained nodes



 copyright©2026 m.patrignani

Computing the repulsive force

- Only adjacent rectangles are taken into account
  - to compute repulsive forces for bucket (i,j) you only have to consider $9 \times O(1) = O(1)$ vertices



 copyright©2026 m.patrignani

Mantaining the data structure

- After each iteration
  - each vertex that moves outside its cell is removed from the old bucket and placed into the new bucket
    - for each vertex it takes O(1) time, if buckets have size O(1)
  - a pointer from each vertex to its bucket is needed to efficiently perform this operation

 copyright©2026 m.patrignani

Fruchterman & Reingold complexity

- The total complexity for the computation of repulsive forces for a single step is

$$\#\text{ of buckets} \times O(1)$$

$$O(n) \times O(1)$$

$$O(n)$$

- Maintenance of the structure can be also performed in $O(n) \times O(1) = O(n)$
- However
  - if the graph is dense, edges may be $O(n^2)$
  - the hypothesis that vertices are uniformly distributed on the drawing area is not realistic

copyright©2026 m.patrignani

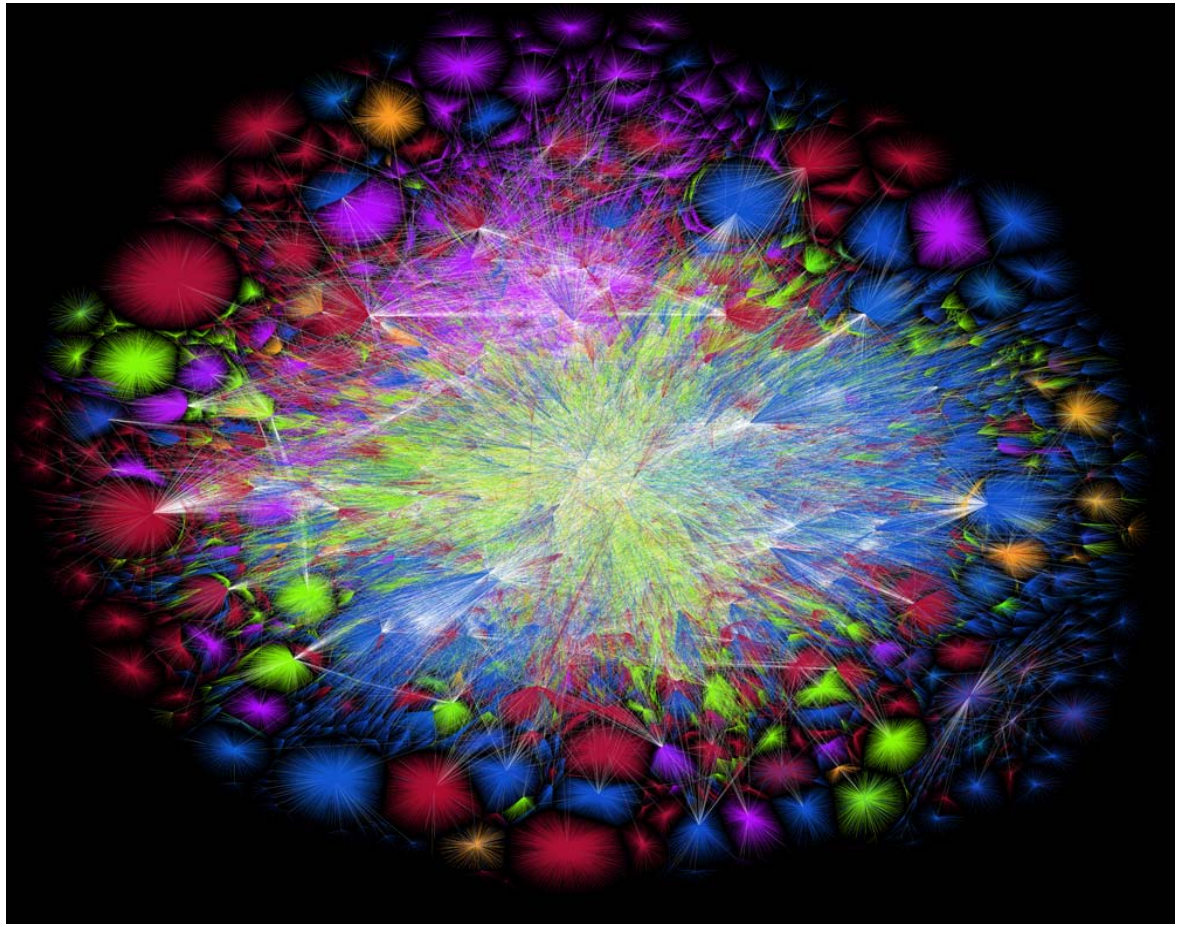
LGL: large graph layout

- A variant of Fruchterman & Reingold due to [Adai et al 04]
- Devised to draw huge graphs with weighted vertices
- Recipe
  - choose a root vertex
    - arbitrarily or based on its centrality in the network
  - starting from the root compute a minimum spanning tree of the graph
    - edges are weighted based on the application domain
  - introduce one level of the tree at a time, granting some time to reach the equilibrium

 copyright©2026 m.patrignani

LGL achievements

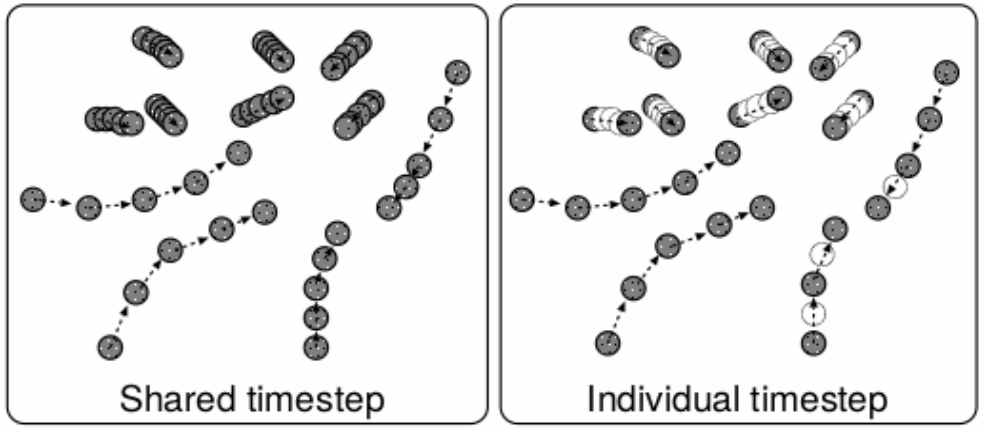
- The “Opte project” using LGL visualized the largest Internet network mapping ever
  - http://www.opte.org
- It examined five million nodes and 50 million edges
- It took 252.68 hours to compute



 copyright©2026 m.patrignani

Matsubayashi & Yamada

- Another variant of Fruchterman & Reingold due to [Matsubayashi & Yamada 07]
- Incorporates the concept of “hierachical individual timestep”
  - introduced to realize the high-accuracy and high speed n-body calculations needed for astrophysical dynamics [Ahmad & Cohen 73]



Picture from [Matsubayashi & Yamada 07]

 copyright©2026 m.patrignani

Individual timesteps

- Shared timestep (Fruchterman & Reingold)
  - even if the force on a node is almost zero, at every timestep the coordinates and the value of the force are updated
- Individual timestep (Matsubayashi & Yamada)
  - every node has its own individual update time
  - if the force on a node is stronger, then the node is updated more frequently

 copyright©2026 m.patrignani

Individual timesteps

- Global time is t
- Each node v_i has
  - a time t_i recording the time when v_i was last updated
  - a timestep Δt_i fixing the interval between two updates
- When $t = t_i + \Delta t_i$ the position of node v_i is updated again as follows

$$p_i(t_i + \Delta t_i) = p_i(t_i) + \Delta t_i \times F(v_i)$$

- Δt_i is also updated
  - instead of setting $\Delta t_i = \frac{1}{F(v_i)}$

    for efficiency reasons they set $\Delta t_i = 2^k$

    with k such that $2^k \leq \frac{1}{F(v_i)} \leq 2^{k+1}$

    - the biggest power of two that is below $\frac{1}{F(v_i)}$

copyright©2026 m.patrignani

Computational complexity

- Let $n_s(t)$ be the number of "syncronized" nodes at time t

$$n_s(t) = \sum_{1}^{n} \begin{cases} 1 & \text{if } t = t_i + \Delta t_i \\ 0 & \text{otherwise} \end{cases}$$

- The complexity of each step of the algorithm is reduced from O(n) to O($n_s(t)$)
  - observe that $n_s(t) < n$

copyright©2026 m.patrignani

Predicted positions of nodes

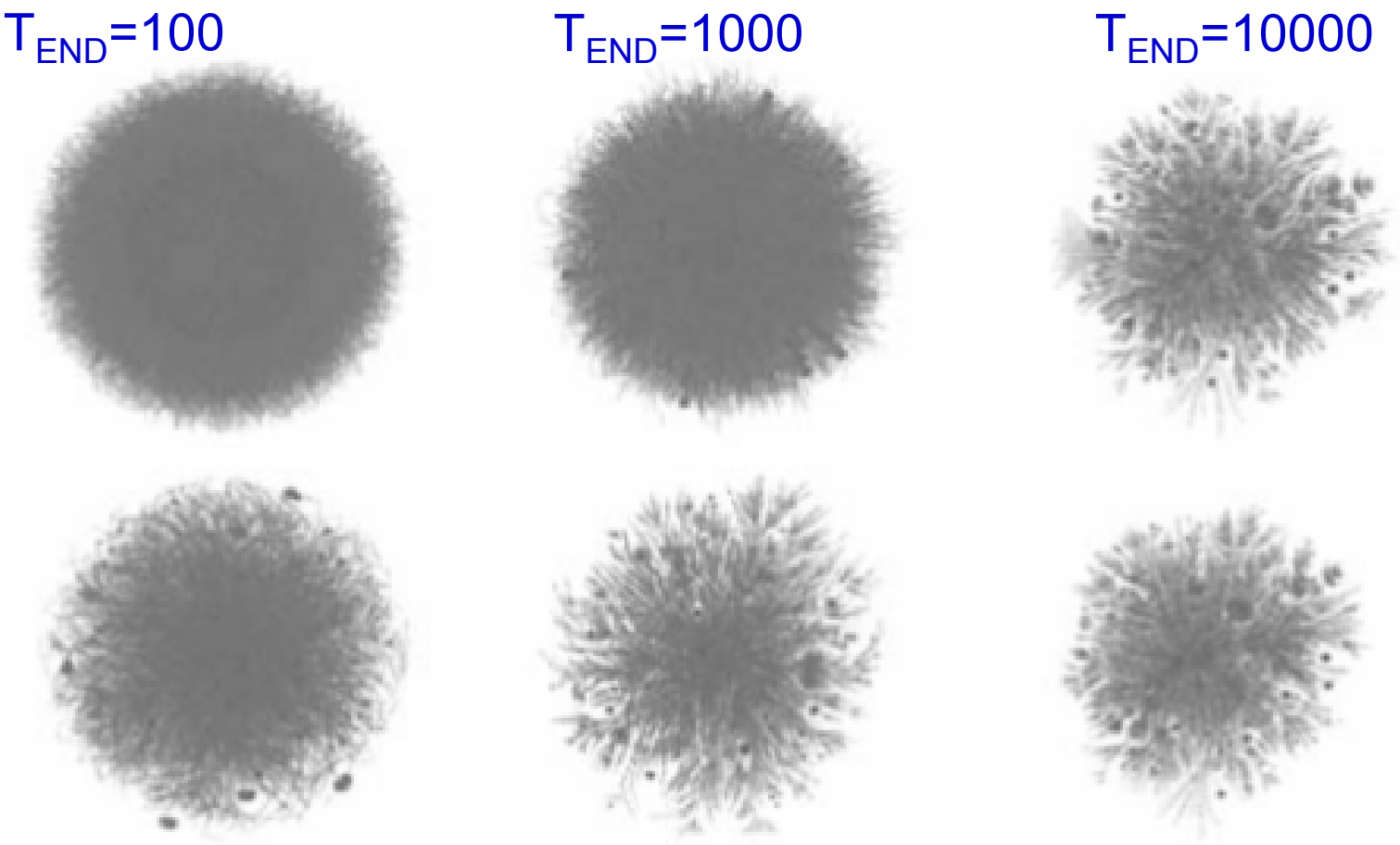
- The force on a node v_i is computed based on the predicted position of each other node v_j
- Suppose v_j at time t_j had position $p_j(t_j)$ and force $F_j(t_j)$
- The predicted position of v_j at time t is

$$\tilde{p}_j(t) = p_j(t_j) + (t - t_j) \times F_j(t_j)$$

 copyright©2026 m.patrignani

Comparison with F&R

- The evolution of the drawings through time

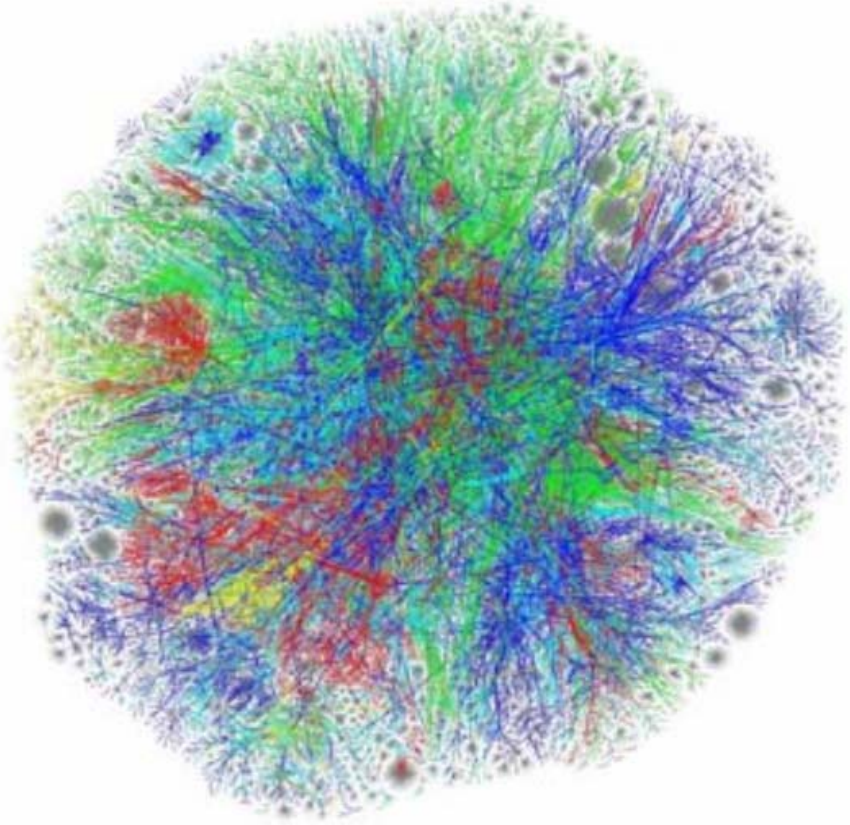


Pictures from [Matsubayashi & Yamada 07]

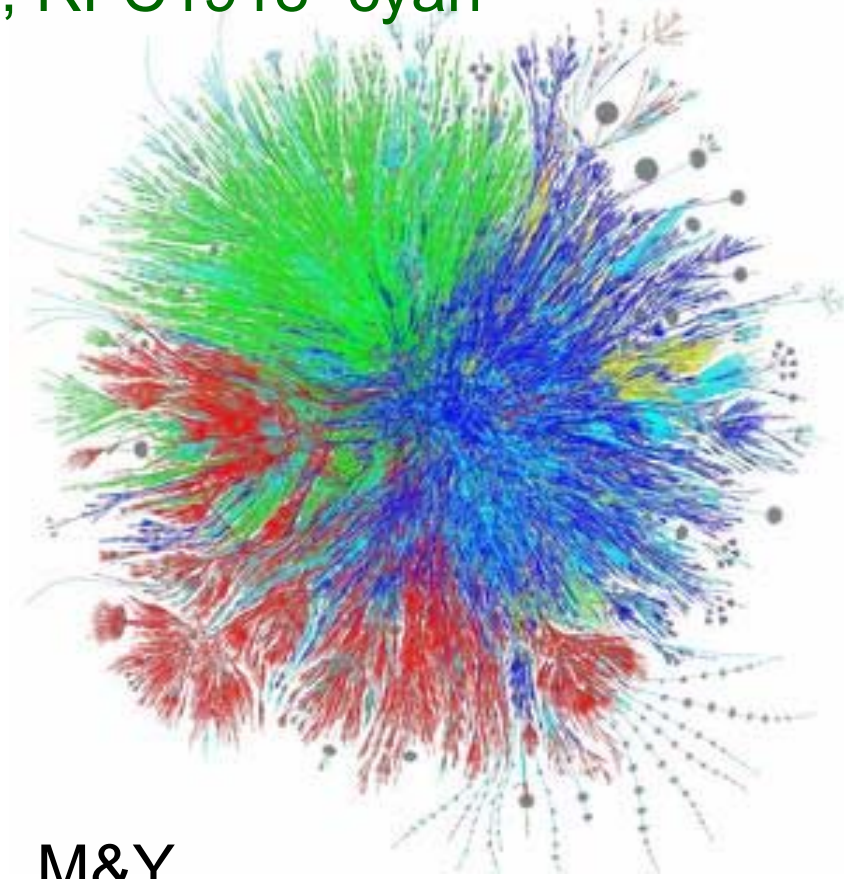
 copyright©2026 m.patrignani

Comparison with LGL

- A drawing of the Internet
  - Colors corresponds to regions: Asia=red, Europe=green, North America=blue, South America=yellow, RFC1918=cyan



LGL



M&Y

copyright©2026 m.patrignani

Barnes-Hut algorithm

- Based on an approximation algorithm developed for astrophysical dynamics by Josh Barnes and Piet Hut [Barnes & Hut 86]
- Proposed for graph drawing by [Quigley & Eades 01]
- Computes repulsive forces in O(n log n)
  - this is apparently more than Fruchterman & Reingold, but is practically quicker

 copyright©2026 m.patrignani

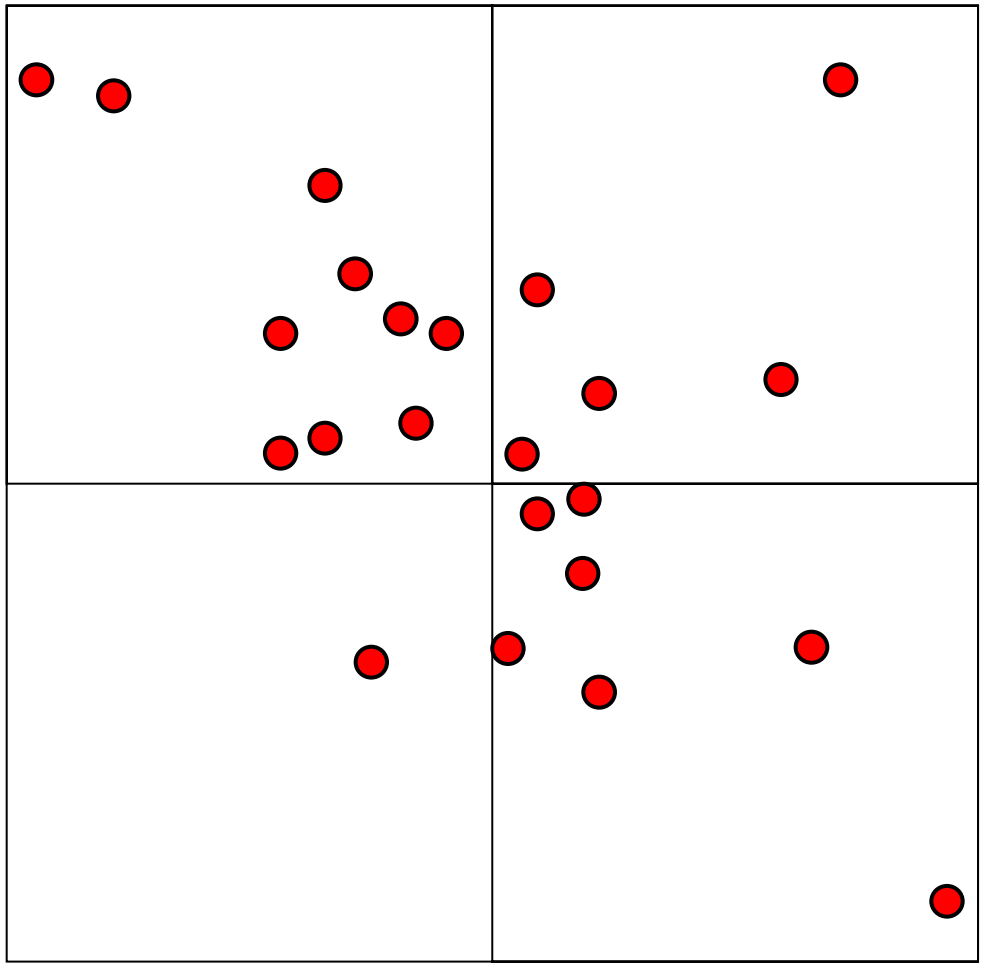
Barnes-Hut algorithm

- Main ingredients
  - recursively subdivides the drawing area into cells of etherogeneous size
  - maintains information about intermediate and final cells into a quad-tree

 copyright©2026 m.patrignani

Barnes-Hut cells

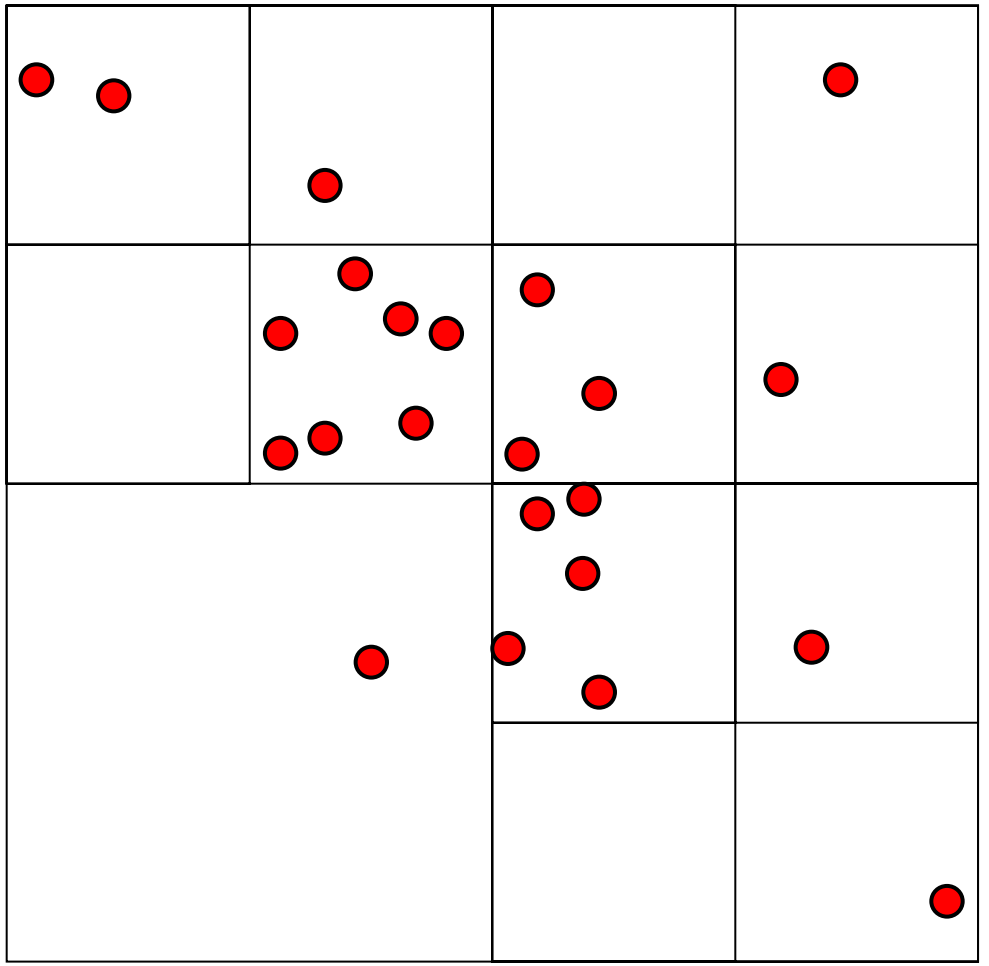
- Recursively subdivide the drawing area into four quadrants
  - stop when each cell contains either zero or one node



copyright©2026 m.patrignani

Barnes-Hut cells

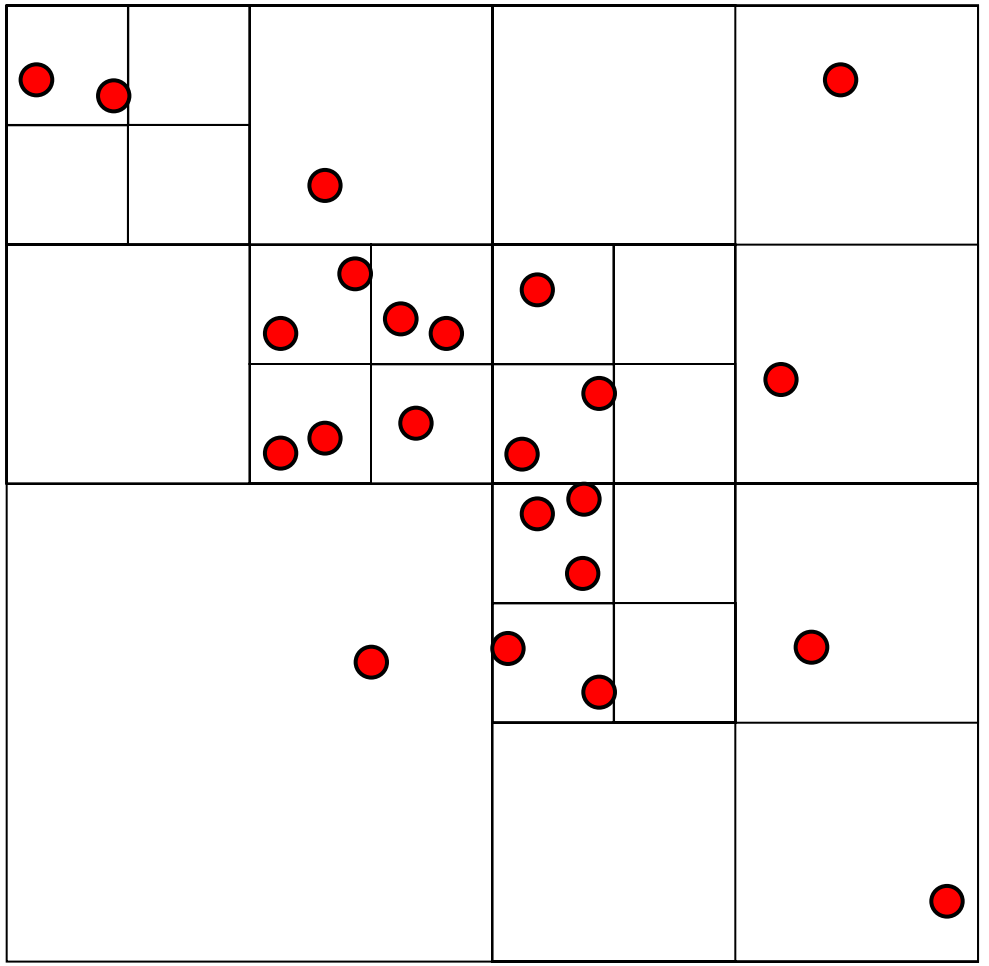
- Recursively subdivide the drawing area into four quadrants
  - stop when each cell contains either zero or one node



copyright©2026 m.patrignani

Barnes-Hut cells

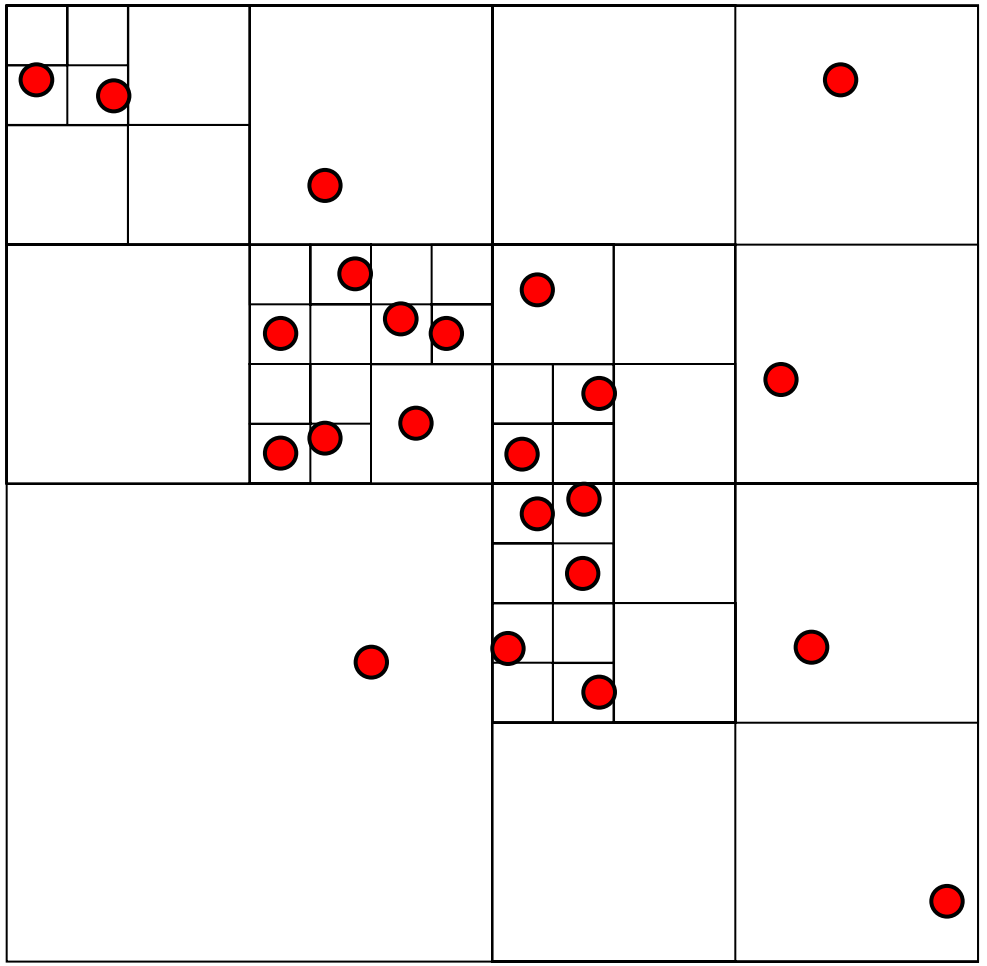
- Recursively subdivide the drawing area into four quadrants
  - stop when each cell contains either zero or one node



copyright©2026 m.patrignani

Barnes-Hut cells

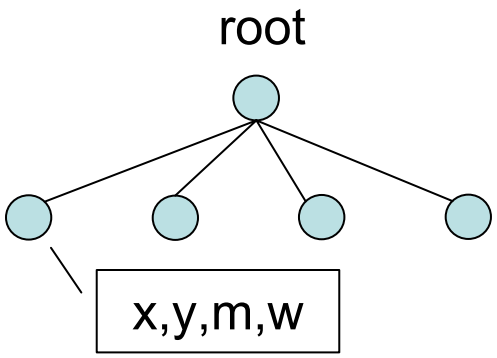
- Recursively subdivide the drawing area into four quadrants
  - stop when each cell contains either zero or one node

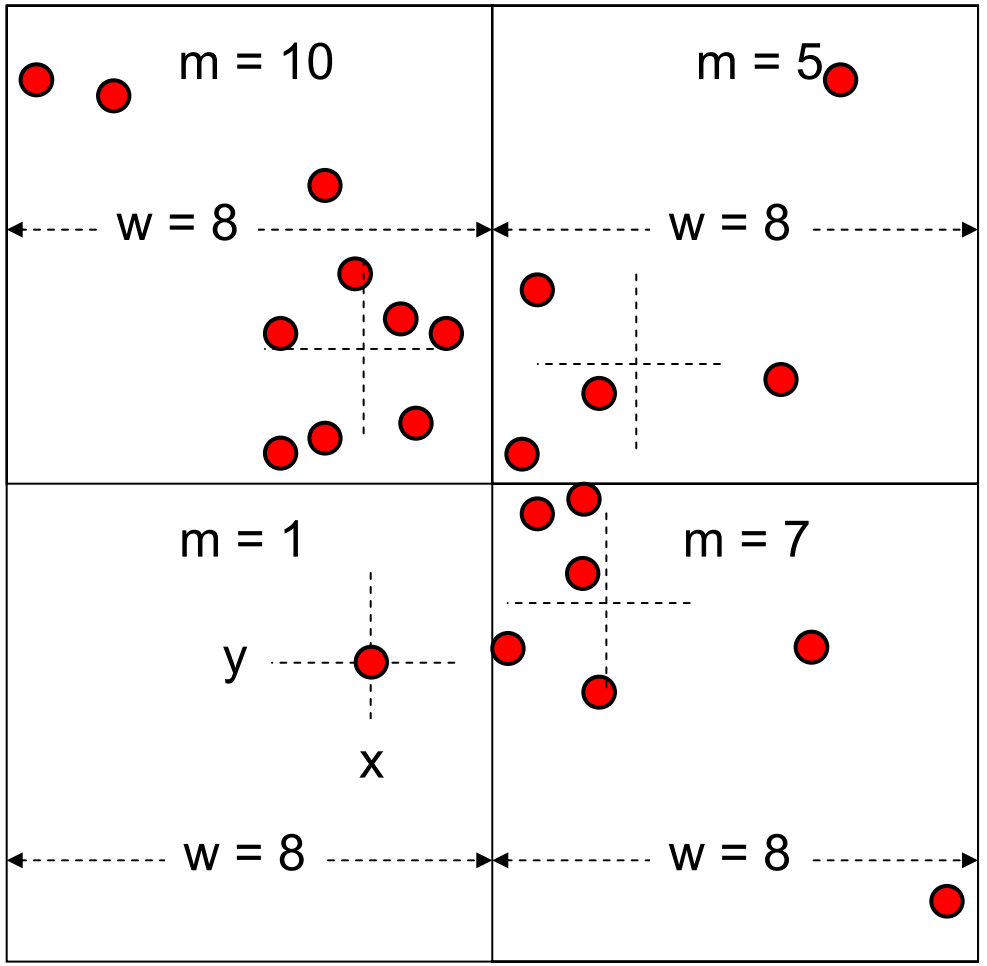


copyright©2026 m.patrignani

Barnes-Hut quad-tree

- Each cell
  - is associated with a node of a quad-tree
  - has a width *w* and a center of mass (*x*,*y*)





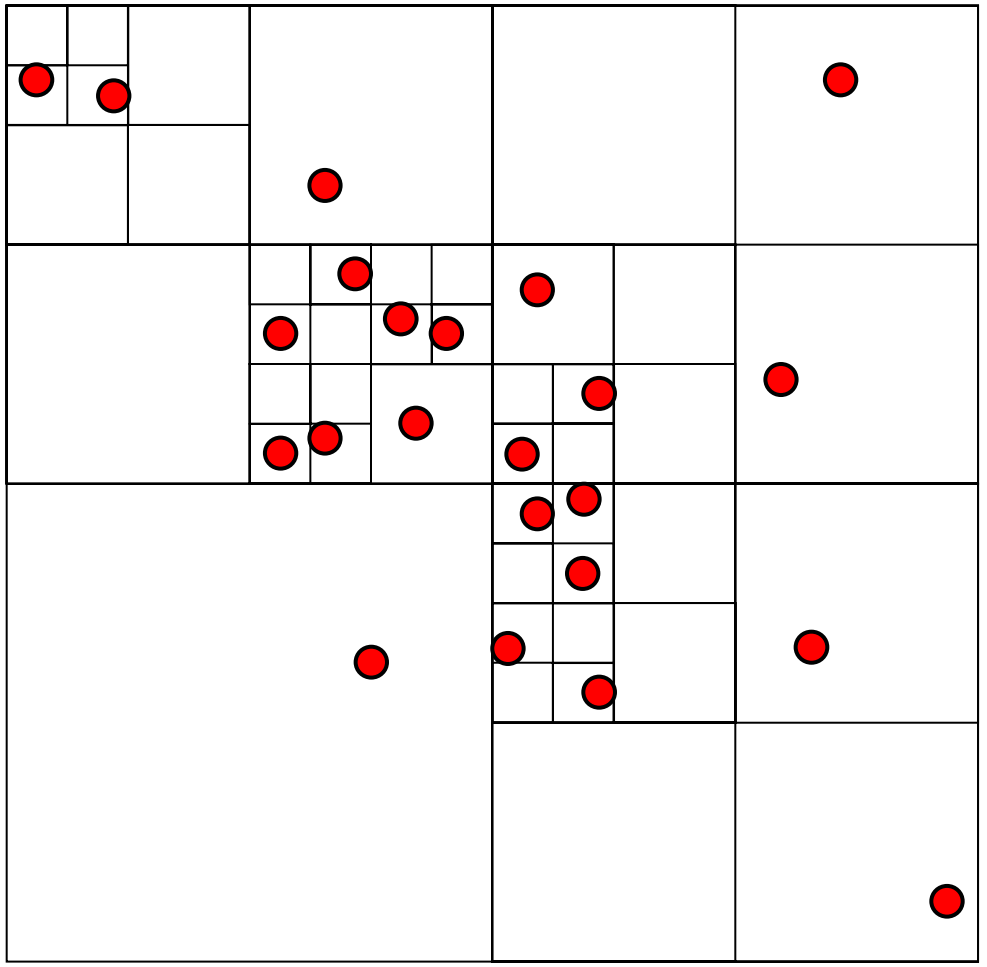
 copyright©2026 m.patrignani

Barnes-Hut repulsive forces

- Consider a vertex *v*
- Compute repulsive forces between *v* and center of mass of some cells
  - only consider cells whose distance *d* from *v* is such that

    $w/d > \theta$

    - you stop descending in the tree when $w \leq \theta\, d$
  - θ is a parameter of the algorithm



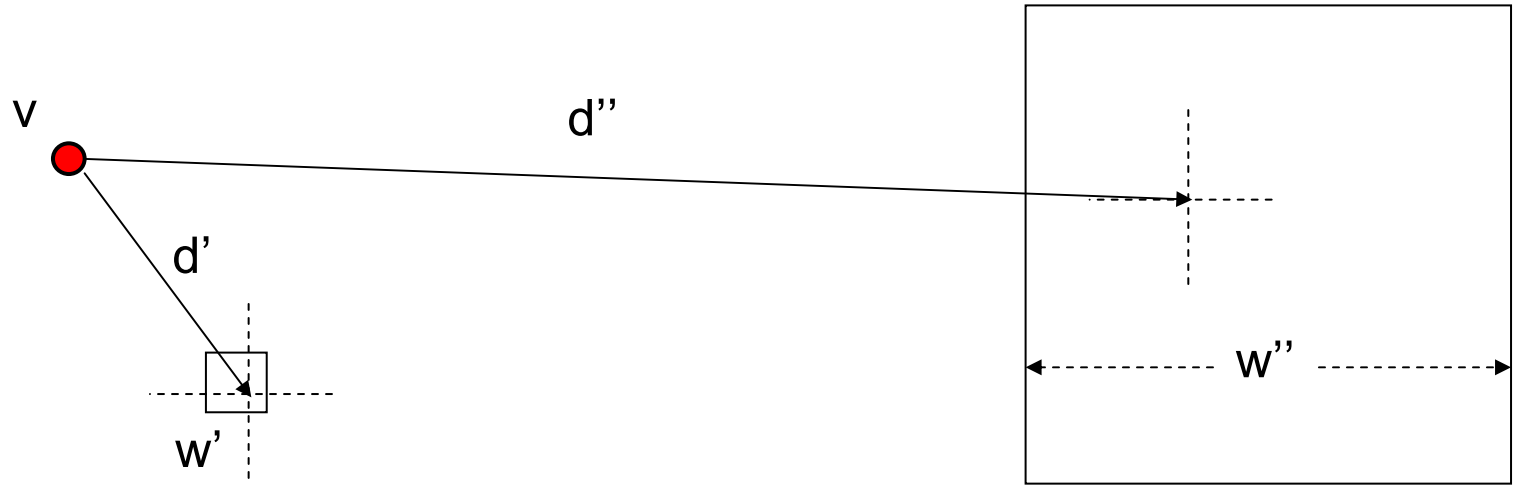
copyright©2026 m.patrignani

Barnes-Hut repulsive forces

- We only consider cells such that

$$w/d > \theta$$

- Intuitively we consider
  - small cells provided they are near
  - far away cells provided they are big



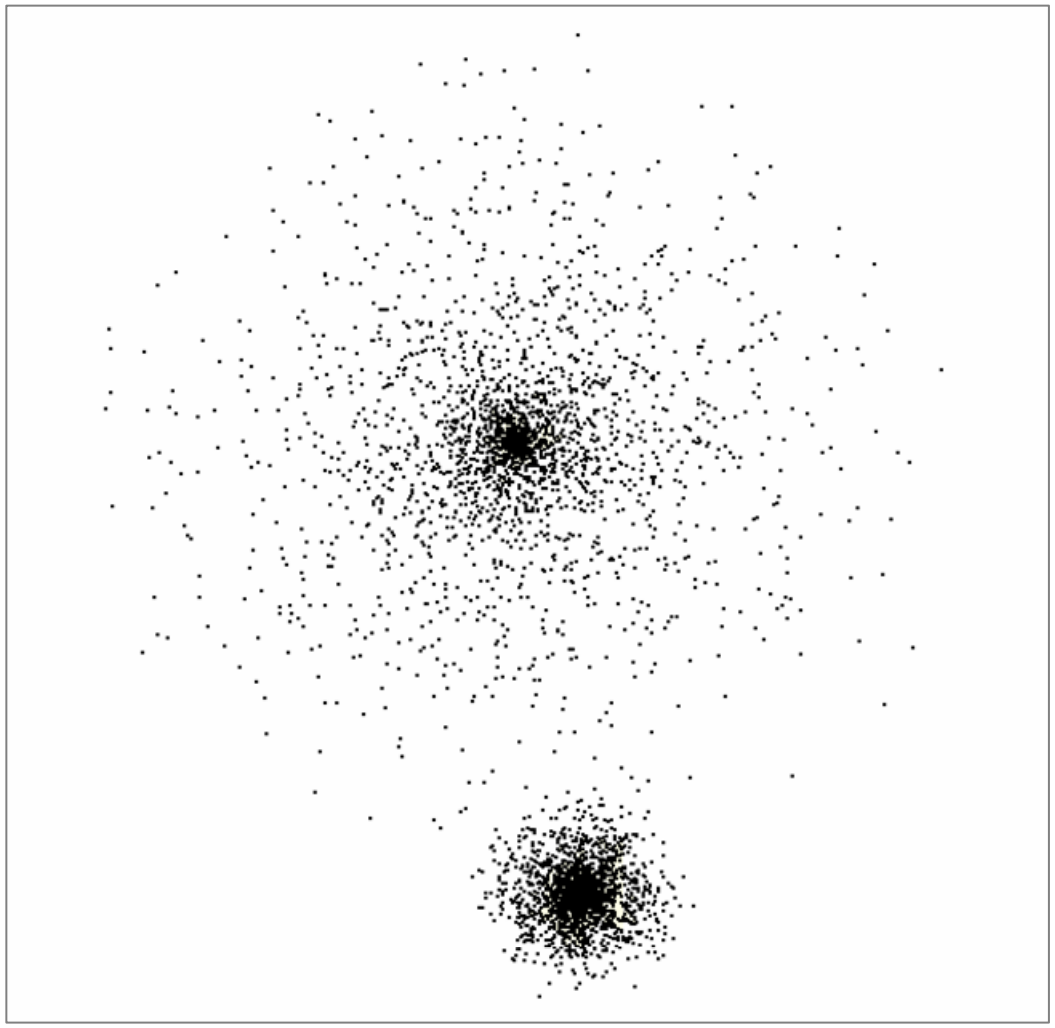
 copyright©2026 m.patrignani

Barnes-Hut parameter θ

- To quickly find the cells to be considered descend the quad-tree and stop at the parents that have at least one child with w/d $\leq \theta$
- The parameter θ determines the accuracy of the simulation
  - larger values of θ increase the speed of the simulation but decrease its accuracy
  - if $\theta = 0$, all leaves of the quad-tree are taken into account and the algorithm degenerates to a traditional spring-embedder

 copyright©2026 m.patrignani

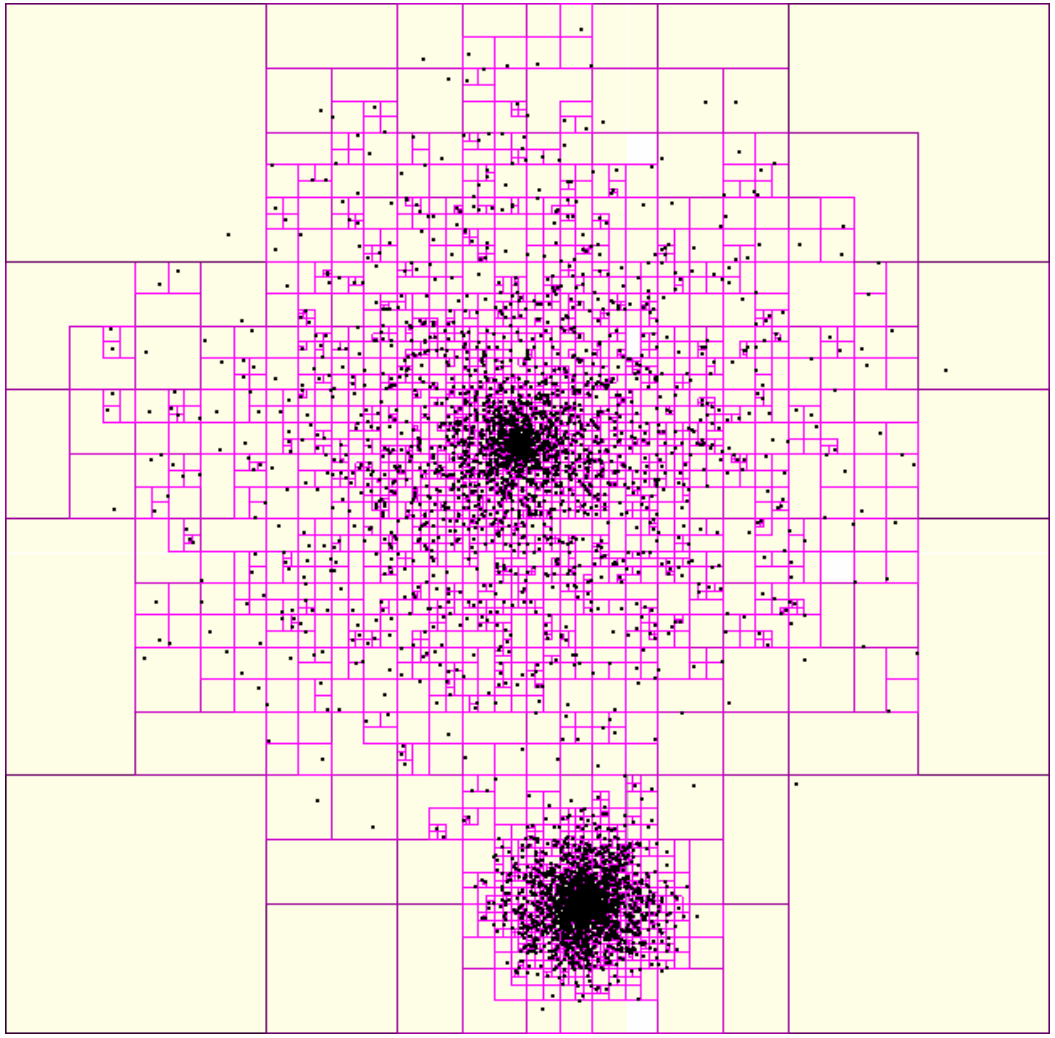
Example: a complex scenario



Picture adapted from Wikipedia

 copyright©2026 m.patrignani

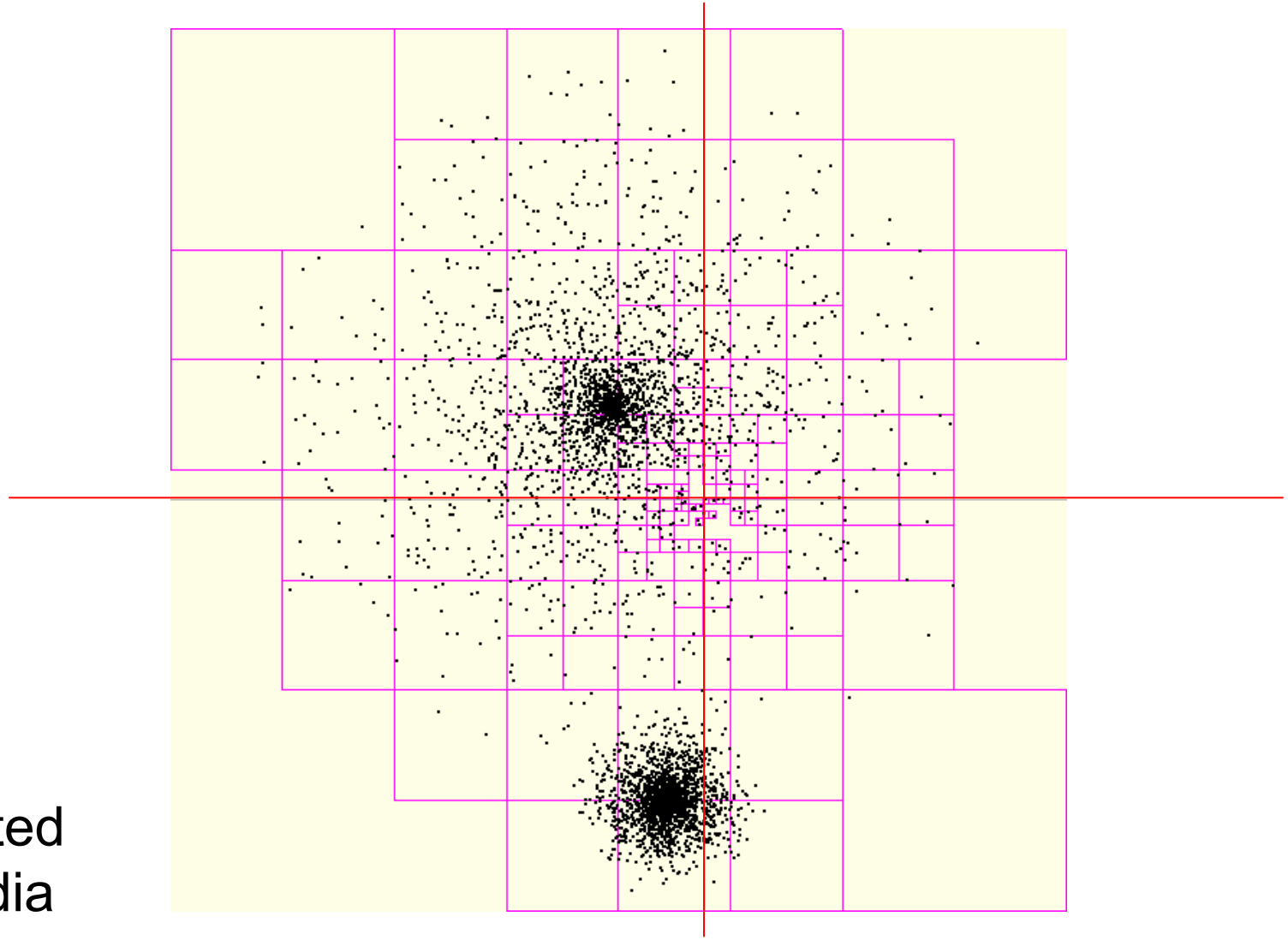
Example: the quad-tree



Picture adapted
from Wikipedia

copyright©2026 m.patrignani

Example: the considered cells



Picture adapted
from Wikipedia

 copyright©2026 m.patrignani

Demos

- Andrei Kashcha's VivaGraphJS
  - https://github.com/anvaka/VivaGraphJS
- Demo at
  - http://www.yasiv.com/graphs#HB/ash219
  - compatible with Chrome only

 copyright©2026 m.patrignani

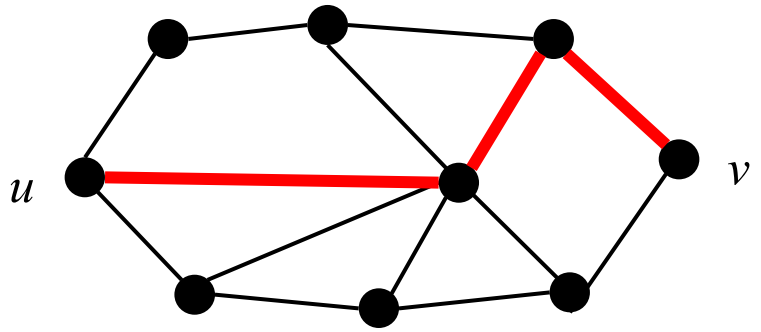
Forces simulating graph-theoretic distances

 copyright©2026 m.patrignani

Definition of graph-theoretic distance

- Let $G=(V,E)$ be a connected graph and $u,v \in V$
- The *graph theoretic distance* $\delta(u,v)$ is the number of edges of the shorter path between u and v

$$\delta(u,v)=3$$



 copyright©2026 m.patrignani

Graph-theoretic distances

- The purpose of this model is computing a drawing of G where the euclidean distance $d(p_u,p_v)$ is proportional to $\delta(u,v)$
  - pioneered in [Kruskal & Seery, 80]
  - independently proposed by [Kamada & Kawai, 89]

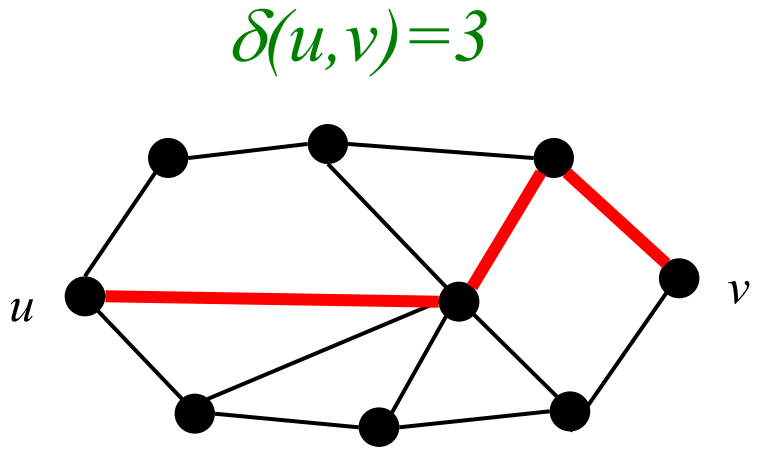
copyright©2026 m.patrignani

Force model

- The force system has forces proportional to $d(p_u, p_v) - \delta(u,v)$
  - it is equivalent to say that the natural length of the springs is assumed to be equal to $\delta(u,v)$

$$F(u) = k_{uv}\left(d(p_u, p_v) - \delta(u,v)\right)$$

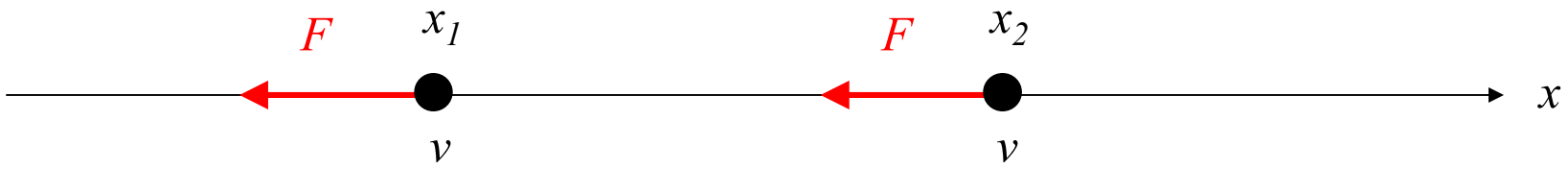
$$\begin{cases} F_x(u) = k_{uv}\left(d(p_u, p_v) - \delta(u,v)\right)\dfrac{x_v - x_u}{d(p_u, p_v)} \\ F_y(u) = k_{uv}\left(d(p_u, p_v) - \delta(u,v)\right)\dfrac{y_v - y_u}{d(p_u, p_v)} \end{cases}$$



copyright©2026 m.patrignani

From force to potential energy

- Suppose to move a point in the opposit direction with respect to a force field



- The work you make is equal to the gained potential energy

$$W = \int_{x_1}^{x_2} F \cdot dx = \eta(x_2) - \eta(x_1)$$

- From which you obtain

$$\eta(x) \approx \int_{x_0}^{x} F \cdot dx$$

copyright©2026 m.patrignani

From force to potential energy

- Hence, the potential energy of the system is the integral of the force
  - and the force is the derivative of the potential energy
- The spring between u and v with the above described model exerts the following force

$$F(v) = k_{uv}\left(d(p_u, p_v) - \delta(u, v)\right)$$

- Which corresponds to the energy

$$\eta = \frac{1}{2} k_{uv}\left(d(p_u, p_v) - \delta(u, v)\right)^2$$

copyright©2026 m.patrignani

Energy function

- The stiffness parameter k_{uv} can be chosen higher for those vertices with lower values of δ
  - that is, $k_{uv}=k/\delta(u,v)^2$ where k is a positive constant
- The energy associated with the interaction between u and v is

$$\eta = \frac{k}{2}\left(\frac{d(p_u,p_v)}{\delta(u,v)}-1\right)^2$$

- The total energy of the whole drawing is

$$\eta = \frac{k}{2}\sum_{u\neq v\in V}\left(\frac{d(p_u,p_v)}{\delta(u,v)}-1\right)^2$$

copyright©2026 m.patrignani

Algorithm

- The algorithm seeks a placement $p_v=(x_v,y_v)$ for each vertex v so to minimize η
- Considering partial derivatives with respect to x and y, the points of minimum satisfy the $2|V|$ *equations*

$$\frac{\partial \eta}{\partial x_v} = 0, \quad \frac{\partial \eta}{\partial y_v} = 0, \quad v \in V$$

copyright©2026 m.patrignani

Iterative approach

- Although the equantions are not linear, it is possible to use an iterative algorithm
  - at each step a single vertex is moved in a position that minimizes the energy of the system
    - all other vertices remain fixed
  - the moved vertex is the vertex on which the stronger force is exherted
    - that is, the vertex that maximizes the quantity

$$\sqrt{\left(\frac{\partial \eta}{\partial x_v}\right)^2 + \left(\frac{\partial \eta}{\partial y_v}\right)^2}$$

 copyright©2026 m.patrignani

Magnetic fields

 copyright©2026 m.patrignani

Magnetic springs model

- Uses magnetized springs
  - proposed by [Sugiyama & Misue, 95]
- Adds a global magnetic field that acts on the springs controlling the direction of the edges
  - of course, it is not required that all the springs are magnetized
- The effect of the magnetic field is summed up with the effects of the electrostatic and spring forces

 copyright©2026 m.patrignani

Spring “magnetization”

- Springs can be magnetized in two ways
  - unidirectional
    - the spring tends to align with the direction of the magnetic field
  - bidirectional
    - the spring tends to align with the magnetic field, but in either direction

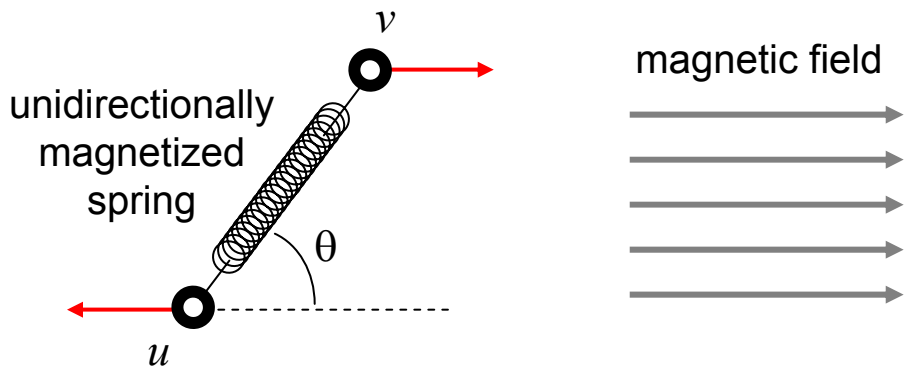
 copyright©2026 m.patrignani

The effect of the magnetic field

- A magnetic field induces a torsion on the springs which tends to align them with its direction
- For a unidirectionally magnetized spring between vertices u and v, the force is proportional to

$$d(p_u, p_v)^{\alpha} \times \sin(\theta)^{\beta}$$

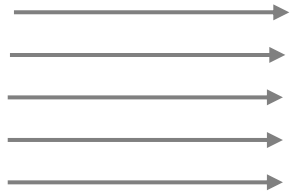
  - where α and β are positive constants

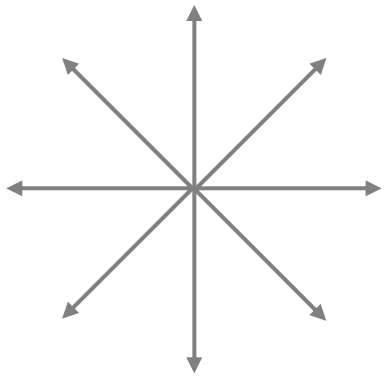


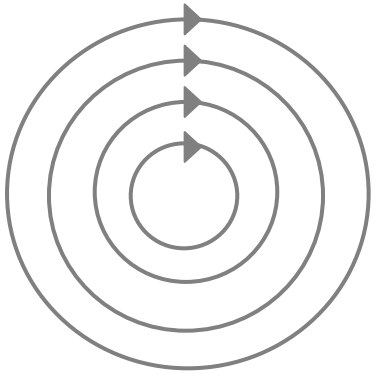
copyright©2026 m.patrignani

Magnetic fields

- Magnetic fields come in three basic types
  - parallel
    - all magnetic forces operate in the same direction
  - radial
    - the forces operate radially outward from a point
  - concentric
    - the forces operate in concentric circles







copyright©2026 m.patrignani

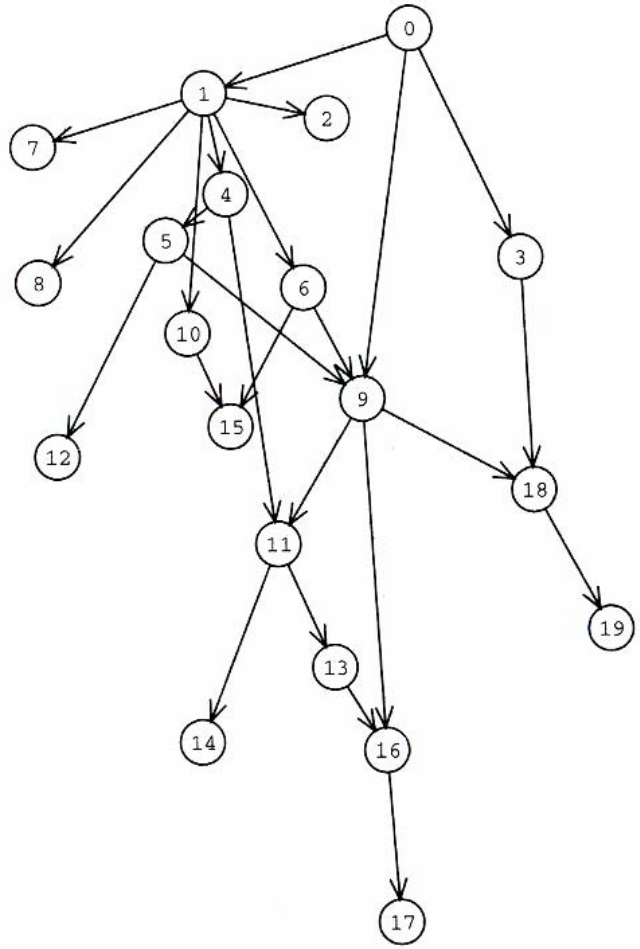
Drawing oriented graphs

- By using magnetic fields the problem of producing layouts of oriented graphs can be addressed
- In fact, we may obtain drawings where most of the edges
  - point toward the bottom
    - magnetic field aligned with the vertical axis
  - point outward
    - radial magnetic field
  - point clockwise
    - concentric magnetic field

 copyright©2026 m.patrignani

Example

- Drawing of a graph with 20 vertices
  - parallel magnetic filed directed towards the bottom of the drawing
  - springs unidirectionally magnetized

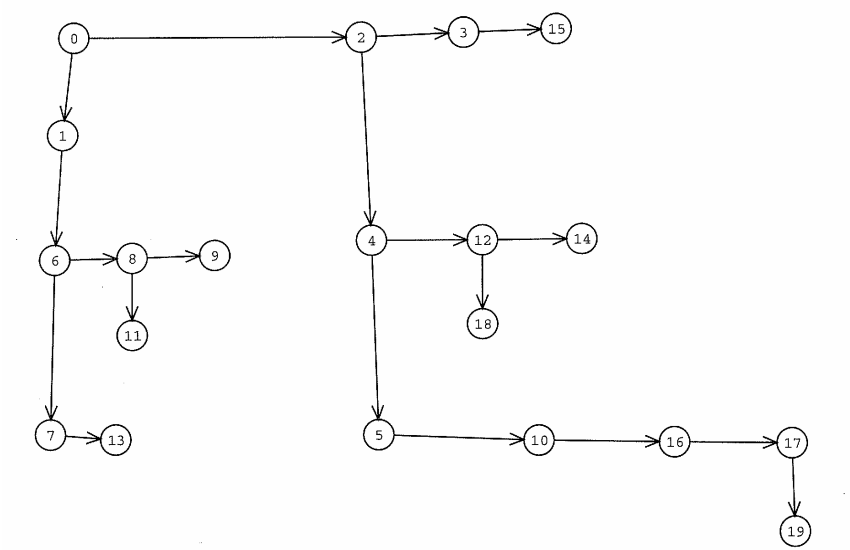


Picture from [Di Battista *et al.*, 99]

copyright©2026 m.patrignani

Combining basic magnetic fields

- The three basic magnetic fields (parallel, radial, concentric) may be combined together
- Example
  - in order to obtain orthogonal drawings we may use a combination of a horizontal and a vertical parallel field



example of drawing obtained by using a combination of horizontal and vertical magnetic fields

Picture from [Di Battista *et al.*, 99]

copyright©2026 m.patrignani

Generic energy functions

 copyright©2026 m.patrignani

Non-continuous functions

- The energy function η of all the models considered till now is a simple and continuous function of the locations of the vertices
- Several aesthetic criteria, as the minimization of crossings among edges, are not continuous
- By allowing discrete energy functions it is possible to broaden the class of aesthetic criteria

 copyright©2026 m.patrignani

Non-continuous aesthetic criteria

- Aesthetic criteria that could interest us are
  - number of edge crossings
  - number of horizontal and vertical edges
  - number of bends along edges

  In addition to the continuous measures already considered
- Recall, though, that some aesthetics conflict with each other
  - it is not possible to optimize with respect to several criteria simultaneously

 copyright©2026 m.patrignani

Generic energy functions

- Generic energy functions with discrete values can be introduced
  - they model crossings, bends, etc…
- The algorithm seeks a configuration that minimizes the total energy
- Such a configuration may be found with several heuristics approaches
  - iterative “follow your nose” strategy
  - simulated annealing
  - genetic algorithm
  - …

 copyright©2026 m.patrignani

Combining together energy functions

- We can use an energy function that linearly combines several energy measures

$$\eta = \lambda_1 \eta_1 + \lambda_2 \eta_2 + \ldots + \lambda_k \eta_k$$

- Among these measures
  - spring energy
  - electrical energy
  - magnetic energy
  - non-continuous ones

copyright©2026 m.patrignani

Algorithm dh96

- For example the following energy model has been proposed by [Davidson & Harel, 96]

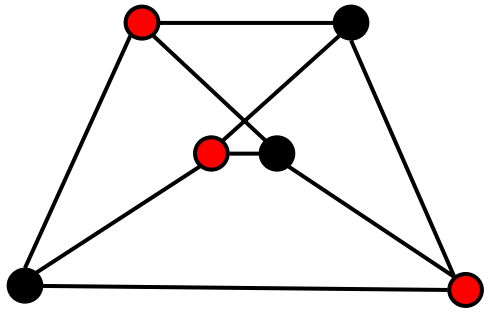
$$\eta = \lambda_1\eta_1 + \lambda_2\eta_2 + \lambda_3\eta_3 + \lambda_4\eta_4$$

- $\eta_1 = \Sigma_{u,v \in V}(1/d(p_u,p_v)^2)$
  - electric repulsion
  - avoids that vertices are too close together
- $\eta_2 = \Sigma_{u \in V}((1/r_u^2)+(1/l_u^2)+(1/b_u^2)+(1/t_u^2))$
  - where r_u, l_u, b_u, and t_u are euclidean distances from the right, left, bottom, and top border
  - aims at keeping the drawing inside the drawing area
- $\eta_3 = \Sigma_{(u,v) \in E}\, d(p_u,p_v)^2$
  - ensures edges do not stretch
- η_4 = # of crossings

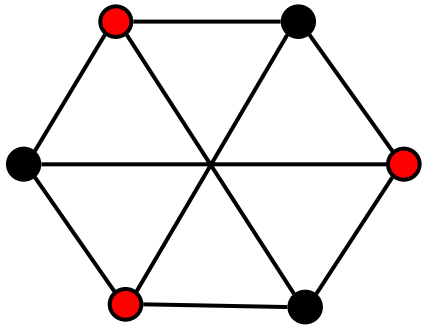
copyright©2026 m.patrignani

Tuning coefficients

- Adjusting the weights λ_i we assign different priority to the various energy functions
  - this allows to recognize major impact to some aesthetics
  - for example, you can draw a $k_{3,3}$…



with the minimum number of crossings

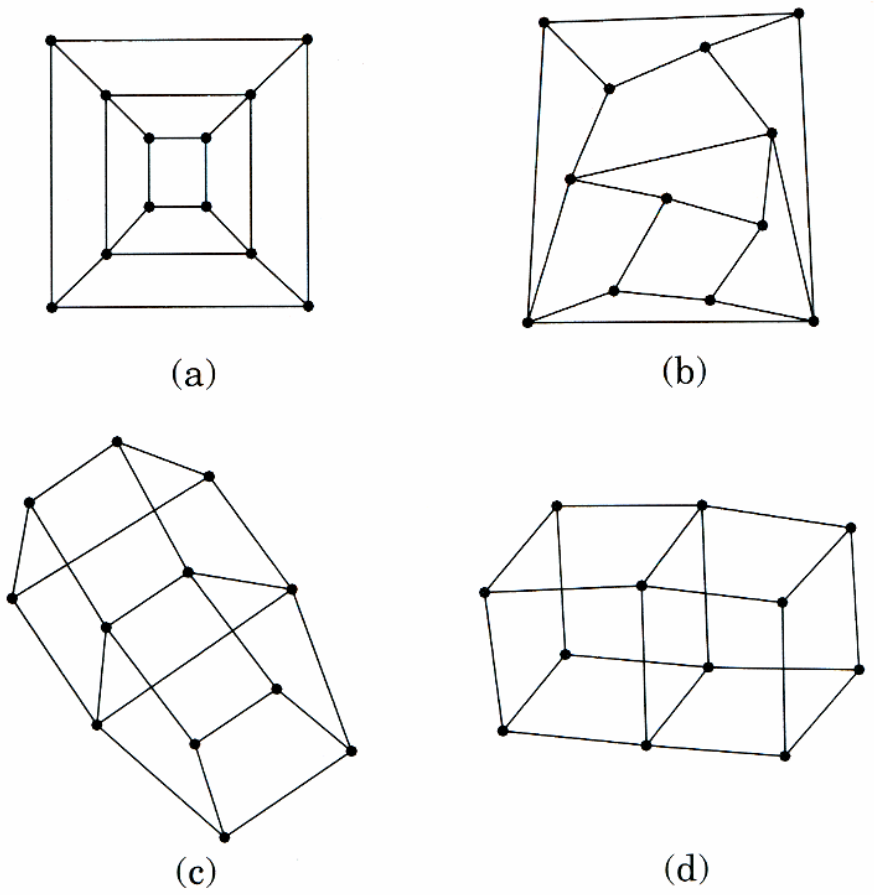


ignoring crossings but privileging symmetries

copyright©2026 m.patrignani

Coefficient fine-tuning: examples

- The input graph is planar and has 12 vertices
  - picture (a), drawn by hand, is planar and symmetrical
- Algorithm dh96 produced drawings (b), (c) e (d)
  - drawing (b) is planar but edges have not uniform length and symmetries are not emphasized



Picture from [Di Battista *et al.*, 99]

copyright©2026 m.patrignani

Adding constraints

 copyright©2026 m.patrignani

Constraining the layout

- Constraints allow us to take into account domain specific needs
- Several constraints are easy to take into account
  - position constraints
  - fixed-subgraph constraints
  - any constraint that can be expressed by forces or energy functions

 copyright©2026 m.patrignani

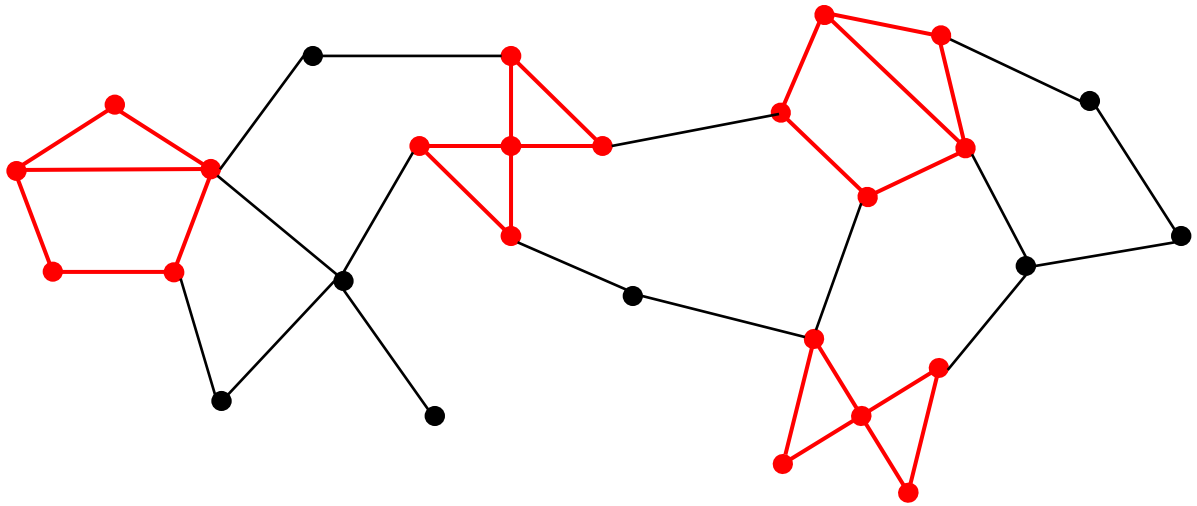
Position constraints

- Constrain the position of a vertex inside a topologically connected region, which may be
  - a single point
  - a horizontal line
    - allows for layered drawings
  - a circle
    - this allows for a set of vertices to be placed near to each other
  - a square
    - for example, the screen of your computer ☺

 copyright©2026 m.patrignani

Fixed-subgraph constraint

- Assign prescribed shape to a subgraph
  - may be translated or rotated, but not deformed
  - handle the subgraph as a rigid body



 copyright©2026 m.patrignani

Constraints expressed by forces

- Using forces you may impose
  - orientation of specific edges
    - magnetic springs
  - clustering of specified set of vertices
    - add a dummy “attractor” vertex for each cluster
    - add repulsive forces among attractors
    - add attractive forces of the vertices of the clusters towards the corresponding attractor
  - alignment of vertices

 copyright©2026 m.patrignani

Concluding remarks

 copyright©2026 m.patrignani

Unexplored variants

- Several attempts have been made to improve the efficiency and effectiveness of force-directed approaches
  - use of randomization in the style of simulated annealing
    - [Frick *et al.*, 95] [Tunkelang 92]
  - use of sophisticated method from numerical analysis tailored on the specific energy functions
  - use of multidimensional scaling (MDS)
    - [Brandes & Pich, 08] [Klimenta & Brandes, 13]
  - use of preprocessing to start from a initial configuration which is not a random one
    - [Fowler & Kobourov, 13]

 copyright©2026 m.patrignani

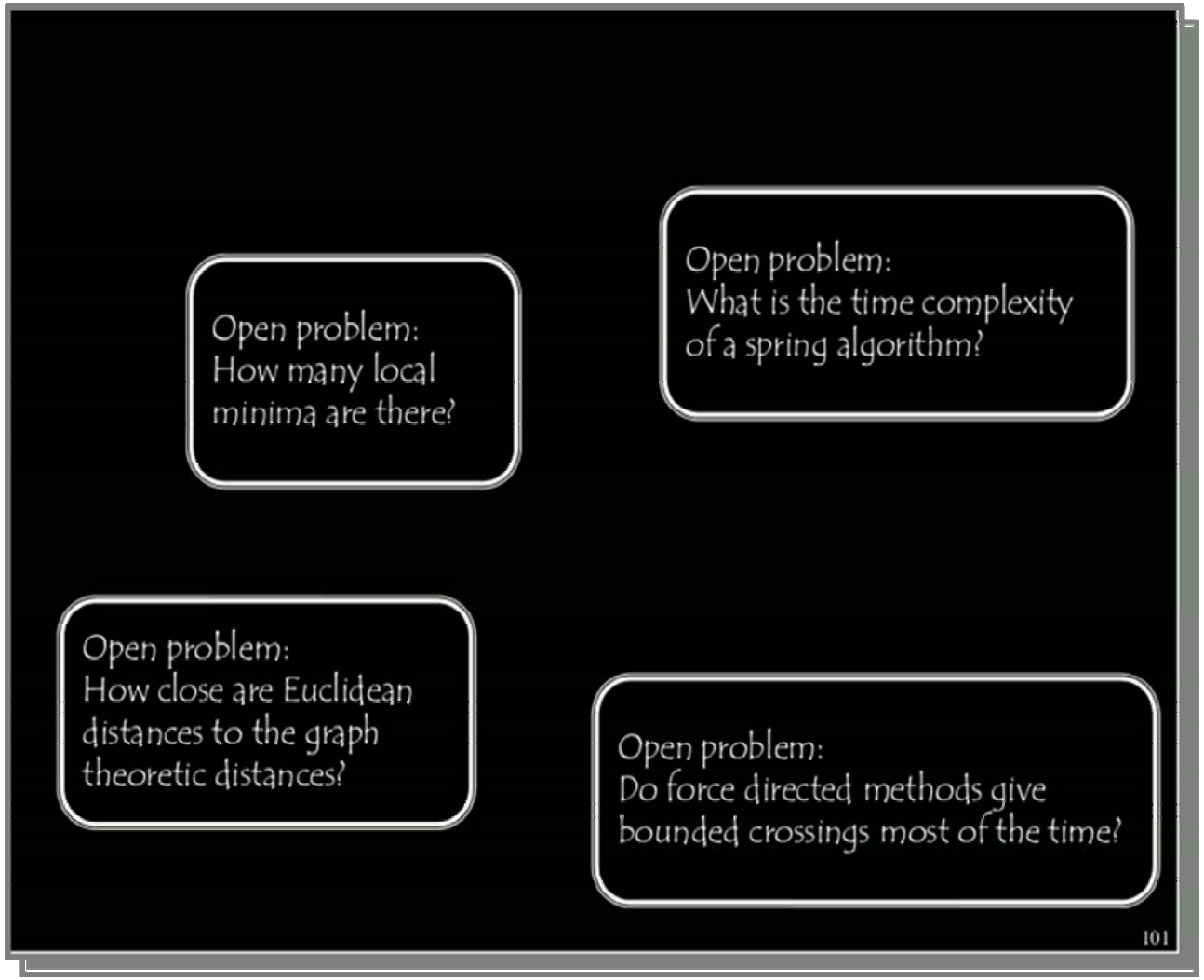
Experimental evaluations

- Force-directed algorithms are heuristics which are best analyzed empirically
- Experimental studies in [Brandenburg *et al.*, ‘96] show that
  - there is no universal winner among the different strains of force-directed methods
  - a good strategy may be that of trying different ones and choosing the one that best suits the specific application
- [Brandes & Pich, ’08] present an extensive experimental study comparing MDS with traditional force-directed methods
  - they show that, if the goal is to represent the distances in a graph, MDS is to be preferred because of its efficiency, reliablility, and even simplicity

copyright©2026 m.patrignani

Open problems (and opportunities)

- There are still some open problems that focus on the basic questions about force-directed approaches



Stolen from Peter Eades's invited talk at GD 2010

copyright©2026 m.patrignani

Bibliography and further readings

- [Adai et al 04] Adai, A. T., Date, S. V., Wieland, S., and Marcotte, E. M.: “LGL: Creating a map of protein function with an algorithm for visualizing very large biological networks”. Journal of Molecular Biology, 2004
- [Ahmad & Cohen 73] Ahmad, A., and Cohen, L.: “A numerical integration scheme for the N-body gravitational problem, Journal of Computational Physics, 1973
- [Barnes & Hut, 86] Josh Barnes and Piet Hut, "A hierarchical O(N log N) force-calculation algorithm". Nature, 1986
- [Brandenburg *et al.*, 96] Franz-Josef Brandenburg, Michael Himsolt, and Christoph Rohrer, “An Experimental Comparison of Force-Directed and Randomized Graph Drawing Algorithms”. Graph Drawing ’95, 1996
- [Brandes & Pich, 08] Ulrik Brandes and Christian Pich, “An Experimental Study on Distance-Based Graph Drawing”, Graph Drawing 2008
- [Davidson & Harel, 96] Ron Davidson and David Harel, “Drawing Graphs Nicely Using Simulated Annealing” ACM Transactions on Graphics, 1996

 copyright©2026 m.patrignani

Bibliography and further readings

- [Di Battista *et al.*, 99] G. Di Battista, P. Eades, R. Tamassia, and I. G. Tollis, “Graph Drawing: Algorithms for the Visualization of Graphs”, Prentice-Hall, 1999
- [Fowler & Kobourov, 13] J. Joseph Fowler, Stephen G. Kobourov “Planar Preprocessing for Spring Embedders” Graph Drawing ‘12, 2013
- [Frick *et al.*, 95] A. Frick, A. Ludwig, H. Mehldau, “A Fast Adaptive Layout Algorithm for Undirected Graphs”, Graph Drawing, 1995
- [Fruchterman & Reingold, 91] Thomas M. J. Fruchterman and Edward M. Reingold, “Graph Drawing by Force-directed Placement”, SOFTWARE—PRACTICE AND EXPERIENCE, 1991
- [Harel & Sardas, 94] David Harel and Meir Sardas, “Randomized Graph Drawing with Heavy-Duty Preprocessing”, AVI 1994
- [Kamada & Kawai, 89] Tomihisa Kamada and Satoru Kawai, ”An algorithm for drawing general undirected graphs”, Information Processing Letters, 1989

 copyright©2026 m.patrignani

Bibliography and further readings

- [Klimenta & Brandes, 13] Mirza Klimenta and Ulrik Brandes, “Graph Drawing by Classical Multidimensional Scaling: New Perspectives”, Graph Drawing ‘12, 2013
- [Kruskal & Seery, 80] J. B. Kruskal, J. B. Seery, “Designing Network Diagrams”, Proc. First General Conference on Social Graphics, 1980
- [Matsubayashi & Yamada 07] T. Matsubayashi, and T. Yamada, “A Force-directed Graph Drawing based on the Hierarchical Individual Timestep Method” International Journal of Electronics, Circuits and Systems, 2007
- [Quigley & Eades] Aaron Quigley and Peter Eades, “FADE:Graph Drawing, Clustering, and Visual Abstraction”, Graph Drawing ‘00, 2001
- [Sugiyama & Misue, 95] Kozo Sugiyama and Kazuo Misue, “Graph Drawing by the Magnetic Spring Model”, Journal of Visual Languages and Computing, Vol. 6, 1995
- [Tunkelang 92] Daniel Tunkelang, “An Aesthetic Layout Algorithm for Undirected Graphs”, M.S. Thesis, 1992

 copyright©2026 m.patrignani